

تقرير المراحل الخمس الاولى من مشروع البرمجة المتوازية

مشروع التجارة الالكترونية هو مشروع يسعى لنقل المتاجر الى العالم الالكتروني ولكن هذا الانتقال فرض مشاكل غير موجودة في عالم الواقع وكان لابد من حلها حتى يرى مثل هذه المشاريع النور وتكسب رضى الزبائن من اشهر الامثلة عليه التنافس على المنتج في عالم الواقع اول من يمسك المنتج يملكه و لكن في العالم الافتراضي قد يوجد تزامن في الطلب بشكل لحظي وهذا يجعل بعض الانظمة تنتقل الى سالب في مستودعها وهذا يعتبر عيب قاتل في تصميم المشروع لذا في هذه المادة كان لابد من تعلم حل هذه المشاكل

خطة بناء المشروع و اختباره

خطة العمل على المشروع كانت بسيطة هي بناء المشروع بشكل تقليدي ثم الانتقال الى حل المشاكل واختبار كم قدم الحل للمشروع من فائدة وكانت هذه الحلول مقسمة الى 10 طلبات .

تم اعتماد اطار عمل Django Rest Framework مع قاعدة بيانات Postgres SQL سبب اختيار اطار العمل هو انه مدعوم بقوة من مجتمع python ويقدم خدمات جاهزة ومكاتب مساعدة كثيرة تسهل العمل على المشروع وتم اختيار قاعدة البيانات لتوافقها العالي مع الاطار فهي الاكثر اعتمادا مع مشاريع Django & DRF

لاختبار المشروع عملنا على استخدام louctus مع استخدام Apidog application كبديل لبوستمان لانه يقدم خاصية اختبار المشروع بالتوازي والاختبار الكثيف ويقدم واجهة سهلة التعامل لانشاء هذه الاختبارات لندخل الان على المشاكل و الحلول ———<<<<

1 . مشكلة التنافس على المخزون

هذه المشكلة واجهناها في مكانين من المشروع المكان الاول هو انشاء حساب جديد ماذا سيحدث عند دخول مستخدمين بنفس الاسم في نفس الوقت للصراحة هي ليست مشكلة بالعادة في اطار العمل الا اذا قررت بناء نظام تسجيل الدخول وانشاء الحساب الخاص بك. نحن اعتمدنا على نظام انشاء الحساب الخاص باطار العمل و الذي يقدم العملية محلولة بحيث انه لا يسمح بوجود مستخدمين بنفس الاسم وذلك داخل قاعدة البيانات مما يجعلنا نتجاوز هذه المشكلة

المشكلة الثانية واجهناها عند محاولة انشاء طلب للمنتجات وتاكيده حيث اذا طلب مستخدمان المنتج في نفس الوقت يمكن ان يتحول التخزين الى السالب و هي ليست مشكلة الا في حال كان الرصيد منخفض جدا فاطار العمل لا يقع في مشكلة ان الطلبين بنفس الكمية والوقت يعتبرهم نفس الطلب لذا كان لا بد من البحث عن حل فعلي اول حل قمنا بتطبيقه هو حل بسيط ولكن يعتبر من اقوى الحلول في حال كان يرغب مدير المشروع بتشديد مبالغ فيه على هذه العملية مع تجاوز نقطة الاداء حيث هذا الحل كان قفل الجدول عن التعديل في حال كان هناك مستخدم اخر يعدل على هذا الجدول هذا الحل قد يبدو قوي ولكنه بقتل نقطة الاداء المتوازي حيث اصبح العمل خط واحد فقط حتى لو كان المشروع كله بعمل بشكل متوازي

ثم انتقلنا الى حل اكثر احترافي وهي تقنية مدمجة مع اطار العمل وهي عملية الاستعلامات اللحظية مع وضع قيود على قاعدة البيانات سنشرح هذا الحل اكثر

- وضع قيد شديد داخل قاعدة البيانات ان كمية المنتج لا تنخفض الى 0 وهذا جعلنا نتجاوز نقطة ان ينزل منسوب المخزن لدينا `stock = models.PositiveIntegerField(default=0)`
- المرحلة الثانية كانت القيد على اطار العمل حيث قمنا بانشاء الطلب المتكامل اي اذا توقف جزء يتراجع عن الكل من خلال `transaction.atomic@` وثم قمنا باعتماد الطلبات الحظية من السيرفر وهي تقنية مدمجة بالاطار تجعله ياخذ استعلامات سريعة جدا في قاعدة البيانات مما يقل خطا معرفة كمية المنتج الموجودة في المستودع `from django.db.models`
- هذه الطبقتين من الحماية تجعل من احتمال حدوث خطأ في المستودع شبه مستحيل وتقدم الاداء المتوازي بشكل سريع و امن

2 . مشكلة ادارة استهلاك الموارد

هذه المشكلة مرتبطة مع المشكلة الخامسة لذا سنذكر جانب منه هنا قمنا بحل هذا المشكلة بطريقة اعتمدنا فيها على ال docker حيث جعلنا هذه الحاوية تستهلك اقل من السيرفر بقليل حيث ان السيرفر لا يصل الى مرحلة الاختناق ويوقف العمل ولكن يبقى السيرفر حي اثناء العمل وهنا تجاوزنا نقطة قتل السيرفر للمشروع ثم عدلنا في استهلاك اطار العمل للموارد من خلال تعديل السيرفر التقائي `python manage.py runserver` الى سيرفر معتمد على `gunicorn` وهو مخصص للبرمجة المتوازية بحيث يتيح تحكم في كم مسار يعمل و كم مدة الطلب وكم يمكن ان يعاد الى اخره من التحكم هذا وقد استخدمنا `nginx` والذي عمل `load balancer` للسيرفر حيث ادار سيرفرين دائمين وسيرفر يضاف عند الحاجة له

وبهذا نقلنا الضغط على المشروع في حالة الضغط من 100% cpu الى 90% , انخفض استهلاك ال ram الى 3GB في حال الضغط وهذا اقل من طاقة السيرفر ب 1GB يعتبر متنفس للسيرفر بحيث لا يقتل السيرفر كل شيء عند امتلى ال ram

3. ارسال طلب الى الخلفية ليعمل

عند طلب شراء المنتج من المستخدم .. يتم ارسال رسالة فورية له مضمونها ("Order is being processed") عوضاً عن الانتظار لفترة طويلة حتى اتمام المهمة .. ويتم تنفيذ المهمة في الخلفية وقد استخدمنا في ذلك Celery و Redis .. وقد استخدمنا ال Redis كونه الوسيط الذي يخزن المهام مؤقتاً وهو من اشهر الانواع بالاضافة للنوع الشهير RabbitMQ ولكن اخترنا Redis لانه اقل تعقيد من ال RabbitMQ وخصوصاً اننا نتعامل مع مشاريع جامعية بسيطة التعقيد الى حد ما غير ان RabbitMQ يتطلب اعداد وادارة اكثر من Redis وهو مناسب للأنظمة العملاقة وهنا بما اننا نعمل على مشروع جامعي بسيط فسيُزيد التعقيد دون فائدة ..

ايضاً اخترنا Redis لانه سريع جداً (in-memory) وبسيط واعداده سهل وسهل الدمج . اما بالنسبة ل celery فهو نظام معالجة المهام غير المتزامنة يعتمد على العميل وهو التطبيق .. ال Broker وهو كما ذكرنا سابقاً مثل Redis .. ال worker هو عملية تعمل في الخلفية تستقبل المهام من ال Broker واستخدمنا celery لانه رفع من الاحترافية في المشروع وهي اداة شهيرة توفر علي عناء التعامل مع ادوات ثانية اقل شهرة منها قد نعاني في التعامل معها وخصوصاً مع ضيق الوقت فكانت حل مثالي واستخدمنا لاحقاً ميزة رائعة من ميزاتها وهي ال Celery Beat لتنفيذ طلبات معينة في وقت محدد

4. جدولة وانشاء تقارير

هنا نقوم بتنفيذ ال Batch processing ونقوم بارسال اشعار لجميع المستخدمين كل اسبوع فحواها ("watch our products") جعلت ال batch = 2 في مرحلة الاختبار فقط .. لكن في الكود الاصلي ابقيته 500 لانه ان كان العدد قليل سيصبح عدد الدورات ضخم واذا كان كبير الذاكرة ستمتلئ ..

واستخدمنا في فكرة ال Batch ال Paginator فهو يقسم البيانات الى pages وكل صفحة تنفذ Query مستقلة .. وسهل الفهم والتنظيم .. وميزته اننا نعرف بالضبط اي Batch يعمل بالضبط .. وهذا على خلاف ال iterator() لو استخدمناه فسنبقى ننفذ مهام كبيرة بدون تقسيم فعلي ..

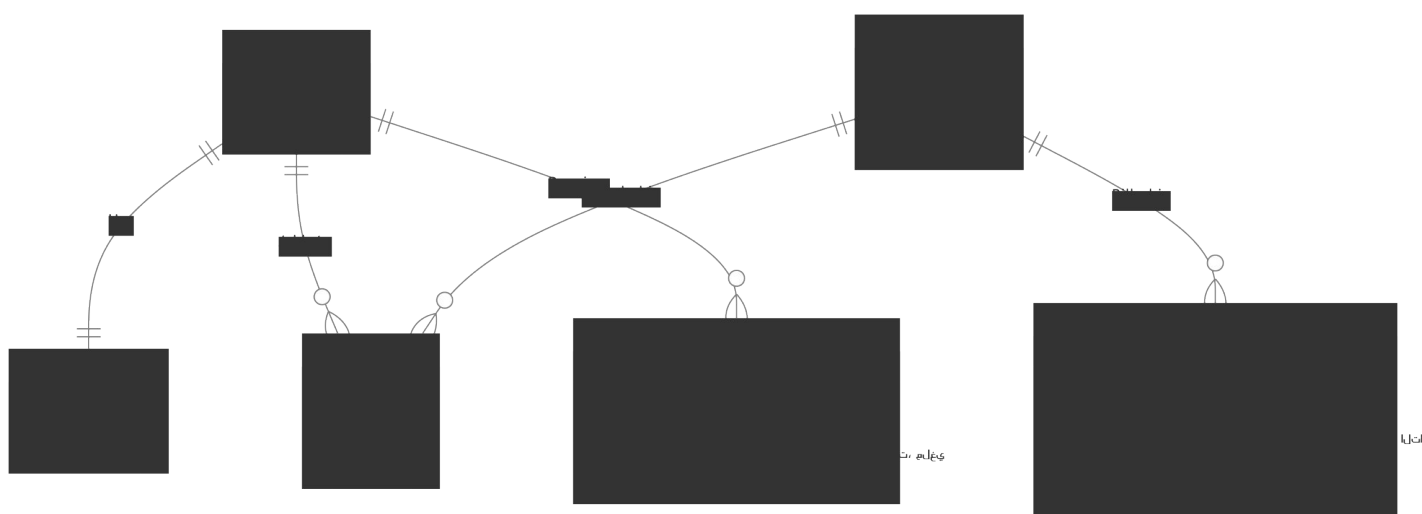
النتيجة :

قمت باستخدام Paginator لتقسيم المستخدمين إلى دفعات صغيرة، ثم استخدمت Celery لإرسال كل دفعة بشكل غير متزامن داخل Tasks مستقلة لتحسين الأداء وتقليل استهلاك الموارد.

5. توزيع الاحمال على سيرفرين

في هذه المرحلة كان يجب وضع المشروع على سيرفرين يعملون بالتوازي لذا اعتمدنا nginx في هذا العمل حيث بنينا سيرفرين مع 16 مسار كل سيرفر يعمل ب 1 core من المعالج مع 1 GB RAM واضفنا سيرفر ثالث للكفاءة يشرف عليه nginx حيث يتم تشغيله عند الضغط وهذا جعل المشروع يعمل بالتوازي داخليا خارجيا وقمنا بجعل قاعدة البيانات طبقة بعد السيرفرات الثلاثة حيث يتصل بها كل السيرفرات ليحصلوا على معلومات متشابه فيما بينهم

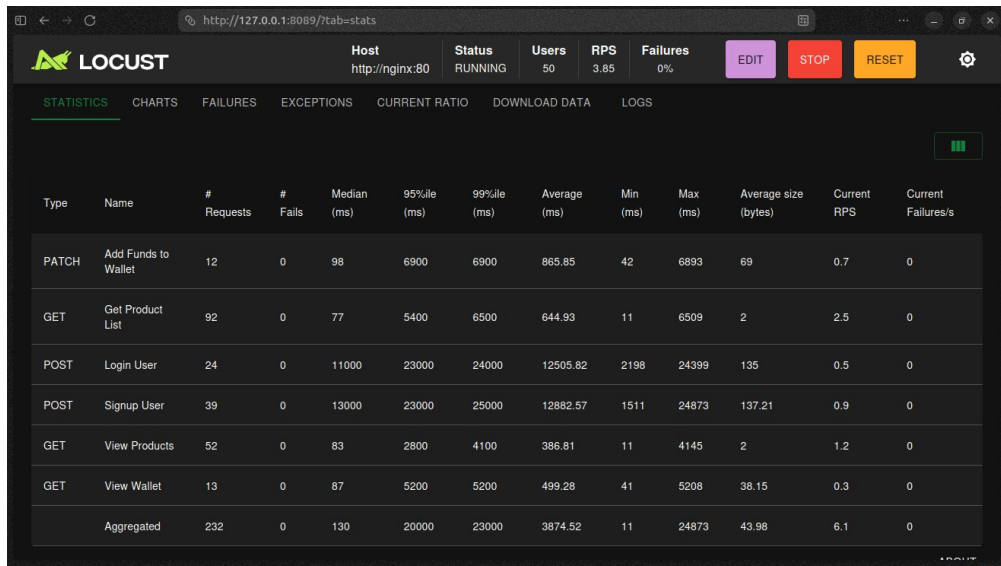
=====



مقارنات الأداء الخاص بالمشروع

1. نبتدء مع اختيار السيرفر الذي يشغل المشروع

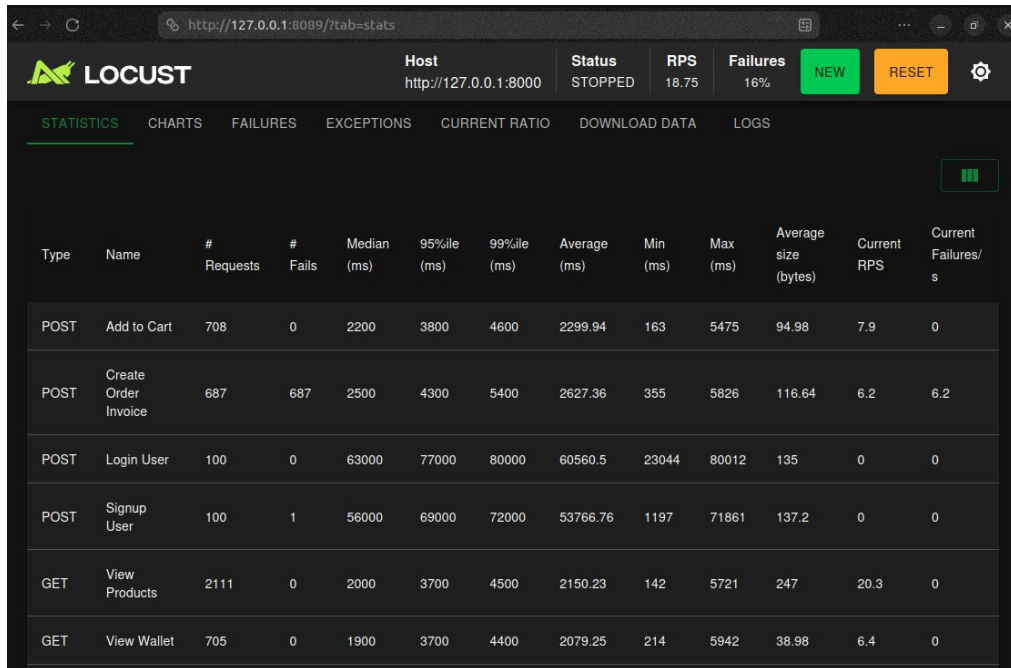
السيرفر الذي اعتمدناه



The screenshot shows the Locust web interface with the host 'http://nginx:80' in a 'RUNNING' status. The 'STATISTICS' tab is active, displaying a table of request metrics. The table includes columns for Type, Name, # Requests, # Fails, Median (ms), 95%ile (ms), 99%ile (ms), Average (ms), Min (ms), Max (ms), Average size (bytes), Current RPS, and Current Failures/s. The data shows various API endpoints like 'Add Funds to Wallet', 'Get Product List', 'Login User', 'Signup User', 'View Products', and 'View Wallet' with their respective request counts and performance metrics.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
PATCH	Add Funds to Wallet	12	0	98	6900	6900	865.85	42	6893	69	0.7	0
GET	Get Product List	92	0	77	5400	6500	644.93	11	6509	2	2.5	0
POST	Login User	24	0	11000	23000	24000	12505.82	2198	24399	135	0.5	0
POST	Signup User	39	0	13000	23000	25000	12882.57	1511	24873	137.21	0.9	0
GET	View Products	52	0	83	2800	4100	386.81	11	4145	2	1.2	0
GET	View Wallet	13	0	87	5200	5200	499.28	41	5208	38.15	0.3	0
	Aggregated	232	0	130	20000	23000	3874.52	11	24873	43.98	6.1	0

السيرفر الاساسي الخاص بالمشروع



The screenshot shows the Locust web interface with the host 'http://127.0.0.1:8000' in a 'STOPPED' status. The 'STATISTICS' tab is active, displaying a table of request metrics. The table includes columns for Type, Name, # Requests, # Fails, Median (ms), 95%ile (ms), 99%ile (ms), Average (ms), Min (ms), Max (ms), Average size (bytes), Current RPS, and Current Failures/s. The data shows various API endpoints like 'Add to Cart', 'Create Order Invoice', 'Login User', 'Signup User', 'View Products', and 'View Wallet' with their respective request counts and performance metrics.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Add to Cart	708	0	2200	3800	4600	2299.94	163	5475	94.98	7.9	0
POST	Create Order Invoice	687	687	2500	4300	5400	2627.36	355	5826	116.64	6.2	6.2
POST	Login User	100	0	63000	77000	80000	60560.5	23044	80012	135	0	0
POST	Signup User	100	1	56000	69000	72000	53766.76	1197	71861	137.2	0	0
GET	View Products	2111	0	2000	3700	4500	2150.23	142	5721	247	20.3	0
GET	View Wallet	705	0	1900	3700	4400	2079.25	214	5942	38.98	6.4	0

سبب اعتماد السيرفر ليس ان ادائه الطلب بالثانية افضل فقد يبدو اقل ولكن السبب هو دعمه لتعدد الخطوط بشكل افضل فقد يعطي مؤشر على قلة اداء ولكن عند جمع التوازي يظهر الاداء افضل ويقل مدة التحميل الفردية فهذا يجعل العمل افضل

2. مشكلة التضارب و الحلول الخاصة بها لقد قدمنا 3 حلول للمشكلة واعتمدنا على واحد

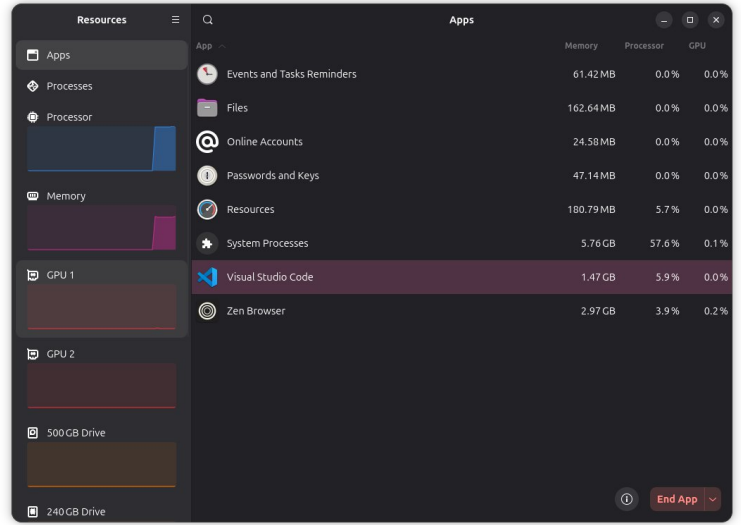
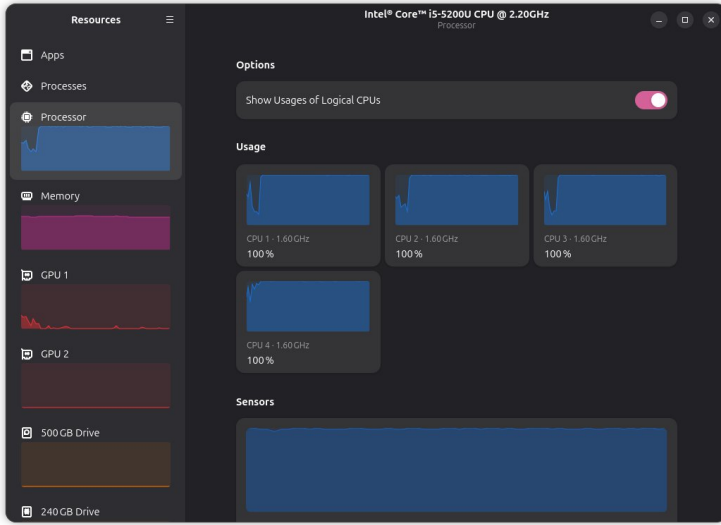
Type	Name	Requests	Fails	(ms)	(ms)	(ms)	(ms)	(ms)	(ms)	size (bytes)	RPS	Failures/s
PATCH	Add Funds to Wallet	50	0	410	900	1100	451.96	175	1123	69.74	2.6	0
POST	Add to Cart	289	0	400	1200	16000	783.1	126	17411	95	11.7	0
POST	Add to Cart (Fix for Invoice)	47	0	410	8400	17000	1365.49	149	16847	95	1.7	0
POST	Create Order Invoice	129	126	500	1100	1500	626.21	164	8865	131.81	4.6	4.6
POST	Login User	36	0	26000	52000	54000	32361.7	18538	53982	135	0.7	0
POST	Signup User	50	0	18000	43000	48000	20046.22	7336	47530	138	0	0
GET	View Products	162	0	390	940	8100	532.29	118	8922	247.02	8.1	0
GET	View Wallet	60	0	330	1400	23000	938.04	121	23316	41.28	2.5	0
	Aggregated	823	126	430	23000	47000	3285.2	118	53982	129.61	31.9	4.6

الحل الاول كان قفل الصف عند التعديل وهذا يعيدنا الى النظام الصفي يعطي نتائج سريعة ولكنها فردية اي خط احادي

Type	Name	Requests	Fails	(ms)	(ms)	(ms)	(ms)	(ms)	(ms)	size (bytes)	RPS	Failures/s
PATCH	Add Funds to Wallet	51	0	240	870	34000	987.25	73	33582	70.24	1.9	0
POST	Add to Cart	253	0	290	2000	20000	1004.34	86	35421	95	8.7	0
POST	Add to Cart (Fix for Invoice)	37	0	260	14000	24000	1737.05	89	23727	95	1.5	0
POST	Create Order Invoice	98	83	400	1900	13000	663.81	102	12946	173.36	3.9	3.9
POST	Login User	40	0	30000	44000	45000	29238.6	15092	44776	135	1.2	0
POST	Signup User	50	1	16000	42000	44000	18285.09	5533	43730	136.4	0	0
GET	View Products	139	0	260	1700	13000	669.5	87	22720	247.32	4.9	0
GET	View Wallet	53	0	230	1500	20000	977.64	98	20306	42.08	2.3	0
	Aggregated	721	84	320	28000	42000	3692.71	73	44776	134.46	24.4	3.9

الحل الثاني كان اعتماد ridce celery لجدولة الطلبات و ادخال الطلبات في حال كان كمية الاستهلاك اقل من المستودع و إيقاف على الدور في حال كان اعلى تم تعطيله بسبب تعقيد زائد و لم يعمل بالكفاءة المطلوبة قد يبدو الاداء اقل ولكنه عمل 4 طلبات دفعة واحدة

ادارة موارد النظام قمنا بذلك على شكلين



الحل الاول تغيير السيرفر لسيرفر اقل استهلاك

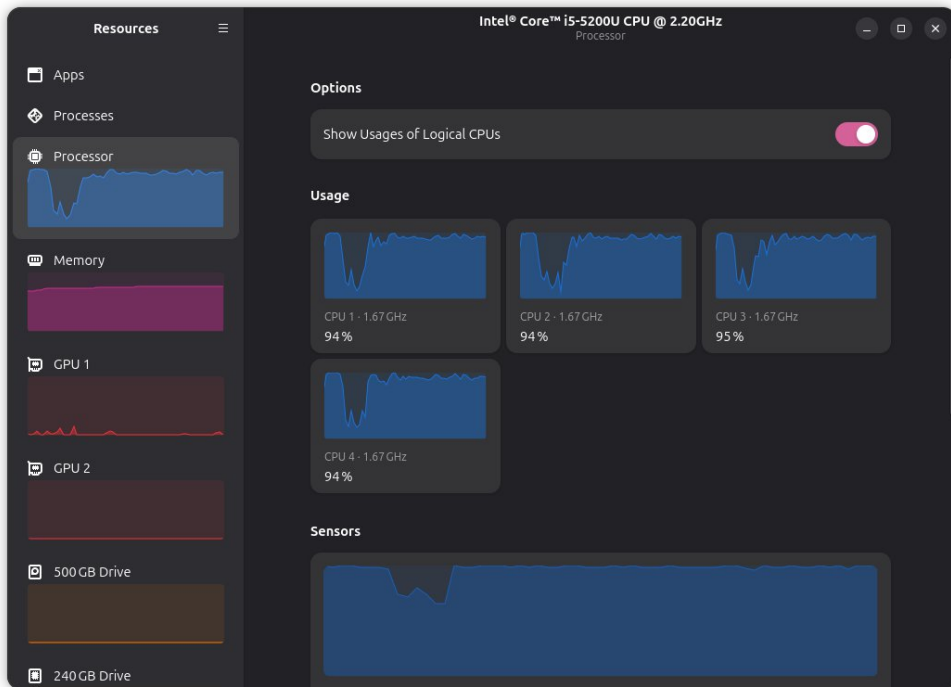
يبدو انه حصل نفس الاداء ولكن الاختلاف الجوهرى هو عدم وجود تعلق في الجهاز حيث مع هذا الاداء الضي استمر لدقائق لم

اضطر الى إيقاف السيرفر

الحال الطبيعي امتلاء المعالج و الرام بشكل مفرط

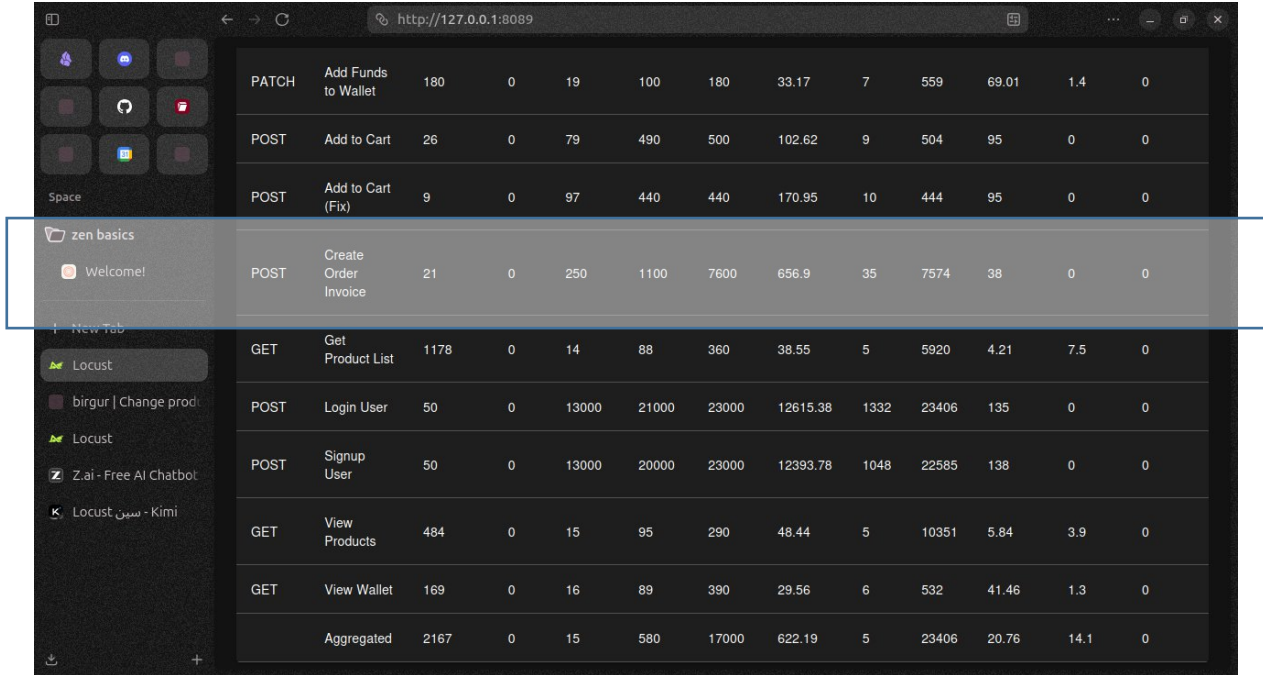
الاداء سيء و يوجد تعليق سيء جدا في الجهاز

الحل الثالث السيطرة على الاداء من خلال دوكر



من الواضح انخفاض اداء المعالج عند الضغط العالي الى 95 بالمية والرام تعتبر اقل استهلاك

الطلب الخامس بناء سيرفر ثاني واعتماد nginx



PATCH	Add Funds to Wallet	180	0	19	100	180	33.17	7	559	69.01	1.4	0
POST	Add to Cart	26	0	79	490	500	102.62	9	504	95	0	0
POST	Add to Cart (Fix)	9	0	97	440	440	170.95	10	444	95	0	0
POST	Create Order Invoice	21	0	250	1100	7600	656.9	35	7574	38	0	0
GET	Get Product List	1178	0	14	88	360	38.55	5	5920	4.21	7.5	0
POST	Login User	50	0	13000	21000	23000	12615.38	1332	23406	135	0	0
POST	Signup User	50	0	13000	20000	23000	12393.78	1048	22585	138	0	0
GET	View Products	484	0	15	95	290	48.44	5	10351	5.84	3.9	0
GET	View Wallet	169	0	16	89	390	29.56	6	532	41.46	1.3	0
Aggregated		2167	0	15	580	17000	622.19	5	23406	20.76	14.1	0

هذا اداء المشروع على الضغط عند اعتماد النهج لحظنا فرق الاداء الفعلي حيث اصبح العمل 16 مسار في كل سيرفر مع 16 في سيرفر الكفاءة ويعتبر الانخفاض كبير جدا

```
use.prod.yml up
app_worker_1-1 | host = request.get_host()
app_worker_1-1 | ~~~~~
app_worker_1-1 | File "/usr/local/lib/python3.11/site-packages/django/http/request.py", line 191, in get_host
app_worker_1-1 | raise DisallowedHost(msg)
app_worker_1-1 | django.core.exceptions.DisallowedHost: Invalid HTTP_HOST header: 'nginx'. You may need to add 'nginx' to ALLOWED_HOSTS.
nginx-1 | 172.20.0.8 - - [18/May/2026:08:23:39 +0000] "POST /api/accounts/signup/ HTTP/1.1" 400 62891
- "python-requests/2.34.0"
app_worker_2-1 | Invalid HTTP_HOST header: 'nginx'. You may need to add 'nginx' to ALLOWED_HOSTS.
app_worker_2-1 | Traceback (most recent call last):
app_worker_2-1 |   File "/usr/local/lib/python3.11/site-packages/django/core/handlers/exception.py", line 55, in inner
app_worker_2-1 |     response = get_response(request)
app_worker_2-1 | ~~~~~
app_worker_2-1 |   File "/usr/local/lib/python3.11/site-packages/django/utils/deprecation.py", line 119, in __call__
app_worker_2-1 |     response = self.process_request(request)
app_worker_2-1 | ~~~~~
app_worker_2-1 |   File "/usr/local/lib/python3.11/site-packages/django/middleware/common.py", line 48, in process_request
app_worker_2-1 |     host = request.get_host()
app_worker_2-1 | ~~~~~
app_worker_2-1 |   File "/usr/local/lib/python3.11/site-packages/django/http/request.py", line 191, in get_host
app_worker_2-1 |     raise DisallowedHost(msg)
app_worker_2-1 | django.core.exceptions.DisallowedHost: Invalid HTTP_HOST header: 'nginx'. You may need to add 'nginx' to ALLOWED_HOSTS.
nginx-1 | 172.20.0.8 - - [18/May/2026:08:23:39 +0000] "POST /api/accounts/login/ HTTP/1.1" 400 62868
- "python-requests/2.34.0"
locust-1 | locust user 14242
```

هذه صورة لعمل السيرفرين مع بعض ولكن لا يتم عرض رسائل من السيرفر الا عند حدوث اخطاء لذلك وضعنا صورة خطأ

الطلب الثالث نقل الاشعار الى الخلفية

```
nginx-1 | 172.20.0.10 - - [18/May/2026:15:24:04 +0000] "GET /api/products/ HTTP/1.1" 200 77 "-" "python-requests/2.34.0"
nginx-1 | 172.20.0.10 - - [18/May/2026:15:24:08 +0000] "GET /api/products/ HTTP/1.1" 200 77 "-" "python-requests/2.34.0"
nginx-1 | 172.20.0.10 - - [18/May/2026:15:24:08 +0000] "POST /api/cart/create/ HTTP/1.1" 201 95 "-" "python-requests/2.34.0"
"
nginx-1 | 172.20.0.10 - - [18/May/2026:15:24:08 +0000] "POST /api/invoices/create/ HTTP/1.1" 202 38 "-" "python-requests/2.34.0"
celery worker-1 | [2026-05-18 15:24:08,939: INFO/MainProcess] Task app.invoices.tasks.process_purchase_order_task[0c317635-41c4-4928-9e83-8bc6250b3d0e] received
celery worker-1 | [2026-05-18 15:24:08,971: INFO/ForkPoolWorker-1] Task app.invoices.tasks.process_purchase_order_task[0c317635-41c4-4928-9e83-8bc6250b3d0e] succeeded in 0.030272439999976086s: {'status': 'success', 'message': 'Order processed successfully', 'invoice_id': 168}
nginx-1 | 172.20.0.10 - - [18/May/2026:15:24:11 +0000] "GET /api/products/ HTTP/1.1" 200 77 "-" "python-requests/2.34.0"
nginx-1 | 172.20.0.10 - - [18/May/2026:15:24:14 +0000] "GET /api/products/ HTTP/1.1" 200 77 "-" "python-requests/2.34.0"
locust-1 | 172.20.0.10 - - [18/May/2026:15:24:14 +0000] "POST /api/cart/create/ HTTP/1.1" 201 95 "-" "python-requests/2.34.0"
"
w Enable Watch
```

هذه اشارة نقله الى الخلفية باستخدام celery Rides

الطلب الخامس صورة للخروج

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\Users\MHD\django_ecommerce> celery -A e_commerce beat -l info
. scheduler -> celery.beat.PersistentScheduler
. db -> celerybeat-schedule
. logfile -> [stderr]@%INFO
. maxinterval -> 5.00 minutes (300s)
[2026-05-18 18:13:20,212: INFO/MainProcess] beat: Starting...
(.venv) PS C:\Users\MHD\django_ecommerce> celery -A e_commerce beat -l info
Could not find platform independent libraries <prefix>
celery beat v5.6.3 (recovery) is starting.
c:\Users\MHD\django_ecommerce\.venv\lib\site-packages\django_q\conf.py:179: UserWarning: Retry and timeout are misconfigured. Set retry larger than timeout, failure to do so will cause the tasks to be retrigged before completion. See https://django-q2.readthedocs.io/en/master/configure.html#retry for details.
warn(
__ _
__ _
LocallTime -> 2026-05-18 18:21:01
Configuration ->
. broker -> redis://localhost:6379/0
. loader -> celery.loaders.app.Apploader
. scheduler -> celery.beat.PersistentScheduler
. db -> celerybeat-schedule
. logfile -> [stderr]@%INFO
. maxinterval -> 5.00 minutes (300s)
[2026-05-18 18:21:01,930: INFO/MainProcess] beat: Starting...
[2026-05-18 18:21:02,030: INFO/MainProcess] Scheduler: Sending due task send-weekly-notifications (app.product.tasks.weekly_notifications)
[2026-05-18 18:22:00,000: INFO/MainProcess] Scheduler: Sending due task send-weekly-notifications (app.product.tasks.weekly_notifications)
[2026-05-18 18:23:00,000: INFO/MainProcess] Scheduler: Sending due task send-weekly-notifications (app.product.tasks.weekly_notifications)
```